# Progress Update: SEBI ICDR Compliance Pipeline

## Rule Extraction - Footnote Handling, Linkage-Aware Amendment Enrichment & Quality Audit

**April 5 – April 25, 2026**

---

## 1. Overview

This round of work built directly on the MVP modularization and bug fixes completed in the April 5 report. Three areas were addressed:

1. **Footnote stripping** - correctly identifying and removing Indian statutory PDF footnotes (both inline citation markers and definition lines) that were being misidentified as regulation numbers by Pass 1.
2. **Amendment linkage enrichment** - upgrading the metadata enricher from a flat PDF-level footnote dump to per-rule, citation-position-aware amendment history.
3. **Output quality audit** - systematic review of `rules_refactored_v5.jsonl` (119 rules, Regulations 4–23) identifying structural, naming, and coverage issues.

Additionally, a strategic direction decision was made on pipeline scope: **extract all 301 regulations first, then build definitions and amendment layers** rather than enriching a partial corpus.

---

## 2. Footnote Stripping

### 2.1 Problem

Indian statutory PDFs embed amendment footnotes in two distinct patterns, both of which caused Pass 1 to misidentify footnote numbers as regulation numbers:

**Pattern A - Inline citation markers** appear mid-sentence, immediately before a `[` bracket containing the substituted text:

```
as on the date of 25[filing] the offer document
is a 26[wilful defaulter or a fraudulent borrower.]
29[(3) If an issuer has issued SR equity shares...]
```

The number is a footnote reference, not a regulation number. The bracket text is the current amended law. Pattern A caused Pass 1 to identify spurious "regulations" 25, 26, 27, 28, 29, 30, 31 and - most damagingly - to parse `29[(3)` as Regulation 29(3) with sub-clauses 29(3)(i) through 29(3)(ix), when the actual regulation is 6(3).

**Pattern B - Footnote definition lines** appear at the bottom of pages, starting at column zero, structurally identical to regulation headers:

```
25 Substituted by the Securities and Exchange Board of India...2019...
26 Substituted by the Securities and Exchange Board of India...2022...
27 Inserted by the Securities and Exchange Board of India...2025...
```

### 2.2 Fix Implemented

Two new regex patterns added to `regex_patterns.py`:

- `INLINE_FOOTNOTE_RE` - matches digits immediately before `[`, preceded by a word/punctuation character (mid-sentence position).
- `FOOTNOTE_DEF_RE` - matches a full footnote definition line: digit(s) at line start, followed by spaces, then one of `Substituted|Inserted|Renumbered|Omitted|Added|Amended`, followed by `by`.

A `strip_footnotes()` preprocessing function runs on raw PDF text **before** any Pass 1 processing. It removes Pattern B lines entirely and strips Pattern A numeric markers while preserving the bracketed amendment text (which is the current legal text). A blank-line cleanup pass follows.

The Pass 1 LLM system prompt (`_PASS1_SYSTEM`) was also updated with generic, regulation-agnostic instructions explaining both footnote patterns and warning the model not to treat bracketed numbers as regulation numbers.

### 2.3 Results

After applying the fix to the pages covering Regulations 4–23:

- Pass 1 no longer identifies footnote numbers 25–31 as regulations.
- `29[(3) If an issuer...]` is correctly parsed as belonging to Regulation 6(3), not Regulation 29(3).
- Bracket text `[filing]`, `[wilful defaulter or a fraudulent borrower.]`, `[(3) If an issuer...]` is preserved in the cleaned text passed to the LLM.
- Footnote definition lines are fully removed from the window text.

---

## 3. Sub-clause Continuation Across Page Boundaries

### 3.1 Problem

Regulation 6(3)(iv) has five lettered sub-clauses (a–e) that were split across a page boundary by the footnote 29/30 definition block. After stripping, sub-clauses `a` and `b` appear on one page, `c`, `d`, `e` on the next - with no visible parent regulation header on the second page. Pass 1 only recovered `b`, missing `a`, `c`, `d`, `e`.

The root cause is a finer-grained version of the continuation page problem: the existing `prev_window_regs` carryover carries top-level regulation numbers (e.g., `6`) but not sub-clause depth (e.g., `6(3)(iv)`).

### 3.2 Fix Implemented

Two additions to the main extraction loop in `llm_extract_rules.py`:

- `prev_window_last_subclause` - tracks the deepest `reg_number` returned by Pass 1 from the previous window, using bracket-character count as a depth proxy.

- `carryover_hint` - a natural-language hint injected into the Pass 1 **user prompt** (not system prompt) when `prev_window_last_subclause` is set, e.g.: "*The previous window ended mid-way through sub-clause 6(3)(iv). Lettered items at the start of this window with no visible parent are continuations of 6(3)(iv).*"

The `_PASS1_SYSTEM` was updated with generic rules for handling pages that open with lettered (`a.`, `b.`, `c.`) or roman numeral (`i.`, `ii.`, `iii.`) items without a visible parent regulation number, directing the model to use the carryover hint to assign them correctly. The Regulation 6-specific example that was previously hardcoded in the system prompt was replaced with a generic pattern description.

---

# 4. Amendment Linkage Enrichment

## 4.1 Previous State

`enrich_rule()` in `metadata_enricher.py` dumped the entire list of PDF-level footnote definitions onto every rule's `amendment_history` field - all 40 footnotes on every one of 119 rules, regardless of which footnote actually amended that rule.

## 4.2 Redesign

`strip_footnotes_with_linkage()` replaces the old `strip_footnotes()`. Before stripping, it scans for inline citation positions (Pattern A) and records:

- `footnote_number` - the citation number found
- `amended_text` - the bracketed text immediately following (the substituted content, up to 200 chars)
- `reg_context` - set to `""` (empty; see note below)

The old `strip_footnotes()` is retained as a compatibility wrapper calling the new function and discarding linkage records.

In the main extraction loop, after each page is processed, linkage records are merged into `accumulated_pdf_footnotes`: for each inline citation found, its `amended_text` is attached to the matching footnote definition entry under a `citations` list.

In `enrich_rule()`, `amendment_history` is now filtered to only footnotes whose `citations[].reg_context` matches the rule's `regulation_number` via prefix logic: a footnote applies to a rule if the rule's regulation number starts with the citation's context, or vice versa (parent inherits child amendments). Empty `reg_context` entries are skipped.

## 4.3 Correctness Fix: `current_reg_context`

The initial implementation passed `current_reg_context=prev_window_last_subclause` to `strip_footnotes_with_linkage()`. This was incorrect: `prev_window_last_subclause` is the deepest clause from the *previous* window, so footnotes found on a page containing Regulation 5(1)(c) text would be tagged with the prior window's context (e.g., `6(3)(iv)`).

The fix is to pass `current_reg_context=""` unconditionally. Linkage records still capture `footnote_number` and `amended_text` correctly; the empty context is safely filtered by `enrich_rule()`.

The prefix matching logic (`startswith`) was verified not to produce false positives for the `regulation_number` format produced by `enrich_rule()` (e.g., `6(3)(ii)`), because the parenthesised format provides natural boundaries.

---

## 5. Output Quality Audit: `rules_refactored_v5.jsonl`

A systematic audit of the 119-rule v5 output (Regulations 4–23) was conducted. Findings by severity:

### 5.1 Structural / Correctness Issues

| Issue | Detail |
|---|---|
| `ICDR_6_3` misattribution | The rule at line 36 with text about "general corporate purposes" and unidentified acquisition targets is not Regulation 6(3). The extractor assigned it the wrong regulation number due to context loss across windows. Requires manual verification against source text. |
| `ICDR_6_3_iv_b` before `ICDR_6_3_iv` | Sub-clause `b` (lines 18–19) appears before its parent `iv` in the file due to different window origins. Sub-clauses `a`, `c`, `d`, `e` are still missing - the continuation page fix is in progress. |
| Duplicate `ICDR_17_proviso` | Lines 95 and 99 share the same `rule_id` but are genuinely different clauses: line 95 is the main exception list, line 99 is the VC fund lock-in sub-condition. Should be `ICDR_17_proviso` and `ICDR_17_proviso_2`. |
| `ICDR_8` ordering | `ICDR_8_proviso` and `ICDR_8_2` appear before `ICDR_8` itself - same root cause as the `6(3)(iv)` ordering issue. |

### 5.2 Naming Issues

| Issue | Detail |
|---|---|
| `ICDR_14_1_proviso_2_that` | The word "that" leaked into the rule ID from "Provided further that". Should be `ICDR_14_1_proviso_2`. The `canonicalize_proviso_markers()` regex strips "Provided further" but leaves "that" when there is no space before it. |
| `_explanation` IDs flagged by malformed checker | The malformed ID checker pattern is too broad - `_explanation` is a legitimate ICDR structural marker. The checker should only flag `_xplanation` (truncated). |

### 5.3 Coverage Gaps

| Issue | Detail |
|---|---|
| 31 rules with empty `maps_to` (26% of output) | Some are legitimately unmappable (procedural rules). Others - `ICDR_6_3_i`, `ICDR_22`, etc. - should have fields. A targeted re-pass with CoT prompting is planned. |

| Issue | Detail |
|---|---|
| `6(3)(iv)` sub-clauses `a`, `c`, `d`, `e` missing | Only `b` recovered. Resolution in progress via carryover hint fix. |

### 5.4 Confirmed Good

- All top-level regulations 4–23 present.
- No JSON parse errors.
- No truncated `rovided` or `xplanation` IDs (proviso canonicalization fix holding).
- All 119 rules have `amendment_history` populated (though currently PDF-level, not per-rule - see Section 4).
- Confidence 0.9–1.0 for 97% of rules.
- All rules `accepted` status.

---

## 6. Strategic Direction: Full Extraction Before Definitions

A decision was made on pipeline scope and sequencing:

**Extract all 301 regulations across all 12 chapters first, then build the definitions and amendment layers.**

Rationale: The reglib design requires `definitions/Core.lean` to be built from a complete picture of what fields all rules actually reference. Extracting Regulation 2 definitions (60+ sub-clauses) before having the full rule corpus would mean building `Core.lean` on a partial foundation - many field types referenced by Chapters III–XI (Rights Issues, FPOs, QIPs, SME IPOs, etc.) would not yet be known. The full ICDR PDF (475 pages, 301 regulations) provides the complete input; the chapter structure is parallel (each chapter has Parts I–IV mirroring Chapter II) so the extraction pipeline generalises directly.

---

## 7. Next Steps

1. **Fix the three correctness issues** identified in the audit (duplicate `ICDR_17_proviso`, `ICDR_14_1_proviso_2_that`, `ICDR_6_3` misattribution) via a targeted post-processing script rather than re-extraction.
2. **Validate the carryover hint fix** on Regulation 6(3)(iv) pages - confirm sub-clauses `a`, `c`, `d`, `e` are recovered.
3. **Run full ICDR extraction** over all 475 pages / 301 regulations using the production pipeline with `--skip-existing` and `--resume` flags.
4. **Targeted re-pass on 31 empty `maps_to` rules** using chain-of-thought prompting.
5. **Integrate per-rule amendment linkage** into enrichment once the full corpus is extracted, using `strip_footnotes_with_linkage()` with correct context assignment.

---

## 8. v6 Extraction Scoring Against Gold Standard

A gold standard JSONL (`gold_standard_regs_4_23.jsonl`, 143 rules, Regulations 4–23) was used to benchmark the v6 output (`rules_refactored_v6.jsonl`, 104 rules). A standalone scoring script

(`scripts/score_extraction.py`) was written to compute Precision, Recall, and F1 with a per-regulation breakdown. Results:

| Metric | v5 | v6 | Delta |
|---|---|---|---|
| Gold standard rules | 143 | 143 | — |
| Extracted (unique IDs) | 119 | 104 | −15 |
| Correctly extracted (TP) | 92 | 100 | +8 |
| Missing (FN) | 51 | 43 | −8 |
| Wrong/extra IDs (FP) | 26 | 3 | −23 |
| Recall | 64.3% | 69.9% | +5.6pp |
| Precision | 78.0% | 97.1% | +19.1pp |
| **F1 Score** | **70.5%** | **81.3%** | **+10.8pp** |

v6's gains are entirely in precision (the anchoring and footnote-stripping fixes eliminated 23 false positives) at the cost of a Reg 6 regression: the tighter anchoring now incorrectly drops 12 valid SR equity share sub-clauses that have no visible parent header on their pages.

**Confirmed clean regulations (v6):** 4, 9, 13, 18, 19, 20, 21, 22, 23.

**Per-regulation gaps in v6:**

| Regulation | Gold | TP | FN | FP | Key Missing Rules |
|---|---|---|---|---|---|
| Reg 5 | 9 | 6 | 3 | 0 | `5_1_explanation`, `5_2_proviso_2`, `5_2_proviso_3` |
| Reg 6 | 23 | 11 | 12 | 0 | `6_3` and all `6_3_iv_*`, `6_3_v` through `6_3_ix` |
| Reg 7 | 13 | 10 | 3 | 0 | `7_3`, `7_3_proviso`, `7_3_proviso_2` |
| Reg 8 | 10 | 3 | 7 | 0 | `8_proviso_3` and all sub-clauses, `8_explanation` |
| Reg 8a | 4 | 3 | 1 | 0 | `8a_explanation` |
| Reg 10 | 10 | 9 | 1 | 1 | Missing `10_1_d_iv`; FP `10_iv` (wrong ID) |
| Reg 11 | 5 | 4 | 1 | 0 | `11_2_proviso` |
| Reg 12 | 2 | 1 | 1 | 0 | `12` (main rule merged into proviso) |
| Reg 14 | 13 | 9 | 4 | 0 | `14_a_proviso`, `14_c_proviso`, `14_4_proviso`, `14_4_proviso_2` |
| Reg 15 | 14 | 10 | 4 | 1 | `15_1_c_proviso`, `15_1_d`, `15_1_iii`, `15_1_explanation` |
| Reg 16 | 6 | 5 | 1 | 1 | `16_1_b_proviso`; FP `16_1_a_proviso` |
| Reg 17 | 10 | 5 | 5 | 0 | `17_b_proviso`, `17_c_proviso`, `17_explanation_i/ii/iii` |

**False positives:** `ICDR_10_iv` (wrong ID for `10_1_d_iv`), `ICDR_15_1_b_proviso` and `ICDR_16_1_a_proviso` (flagged for manual review — may be valid 2025 amendment insertions not yet in the gold standard).

The scoring script (`scripts/score_extraction.py`) produces this breakdown automatically and exits with code 1 if F1 falls below a configurable target (default 90%).

---

## 9. v7 Improvement Plan

**Target:** F1 > 90%, Recall > 85%, Precision > 95%. **Projected result:** TP ≈ 136, FN ≈ 7, FP ≈ 0–1 → F1 ≈ 93–95%.

Seven root causes account for all 43 missing rules, confirmed by analysis of `pass2_pre_judge.jsonl` and `coverage_summary.json` from the v6 run.

### Fix 1 — Reg 6(3) Continuation Pages (12 rules)

**Root cause:** The SR equity share sub-clauses in `6(3)` span pages with no visible `6.` header. v6's anchoring rejects rules labelled `6(3)` on windows where the regex only detects `7` — this is why v6 regressed on Reg 6 compared to v5 (-7 TP on this regulation).

**Two-part fix:**

- **Part A — Grandparent carryover hint.** The existing carryover reports the deepest sub-clause seen (e.g., `6(3)(iv)`). Change it to report the grandparent (`6(3)`) so that lettered items `a.`–`e.` and roman numerals `v.`–`ix.` at the start of a window can be assigned to the right parent level.
- **Part B — Anchoring exemption.** In the anchoring/drop step, add a check: if a carryover hint is active and the rule's top-level regulation matches the carryover context, allow it through regardless of `detected_regs`. This prevents over-strict anchoring from discarding valid continuation-page rules.

### Fix 2 — Reg 7(3) Amendment-Inserted Sub-Regulation (3 rules)

**Root cause confirmed:** Zero entries for `ICDR_7_3*` in `pass2_pre_judge.jsonl` — Pass 1 never identifies them. After footnote stripping, the text reads `[(3) The amount for general corporate purposes...]` with a leading `[` at line start that blocks `REG_HEADER_RE` from matching.

**Fix:** Add `INSERTED_SUBREG_RE = re.compile(r"^\[(\d+)\s*\(", re.MULTILINE)` to `regex_patterns.py`. In `pre_identify_regulations()`, detect these patterns and surface the sub-regulation number to Pass 1. Update `_PASS1_SYSTEM` with an explicit instruction: text beginning `[(N)` at line start is a sub-regulation inserted by amendment; extract it as `"{parent_reg}(N)"`.

Applies equally to the three missing Reg 5 amendment insertions (`5_2_proviso_2`, `5_2_proviso_3`).

### Fix 3 — Reg 8 Proviso-3 Block (7 rules)

**Root cause:** The "holding period shall not apply" block (`8_proviso_3` and its six sub-clauses) is missing due to two compounding issues: (a) anchoring drops rules labelled `8_proviso_3` on pages where only `8A` is detected in `detected_regs`, and (b) `ICDR_8_proviso_2` text is truncated at 300 chars so the nested proviso-3 block is never seen by Pass 2.

**Fix A — Base-reg expansion:** When building `allowed_regs`, always include the numeric base of any alphanumeric regulation: `"8A"` → also allow `"8"`. One function, one call site.

**Fix B — `8_explanation`:** Update `_PASS1_SYSTEM` to treat `Explanation.` blocks as separate structural elements with `reg_number = "{parent}(explanation)"`.

### Fix 4 — Deep Provisos Truncated at 300 Chars (12 rules)

**Root cause confirmed by data:** Multiple parent clauses in `pass2_pre_judge.jsonl` hit the 300-char cap mid-sentence — `ICDR_14_b`, `ICDR_14_c`, `ICDR_14_4`, `ICDR_14_proviso_2` are all at exactly 300 chars and cut off before their nested `Provided that` text. This is why `14_a_proviso`, `14_c_proviso`, `14_4_proviso`, `14_4_proviso_2` are entirely absent from Pass 2 output.

Same truncation causes the missing rules in Regs 15, 16, and 17.

**Two-part fix:**

- **Part A:** Raise all Python-side `clause_text[:300]` / `clause_text[:500]` caps to `clause_text[:1200]`. Update the Pass 1 user prompt template accordingly.
- **Part B:** Inject a `NESTED_PROVISO_INSTRUCTION` into the Pass 2 prompt: if `clause_text` contains one or more `"Provided that"` sub-clauses, emit them as separate JSON objects (suffixed `_proviso`, `_proviso_2`, etc.), never merged into the parent rule's text.

### Fix 5 — `icdr_structure.json` Chapter Assignment (metadata)

Regulations 5(1)(a–d), 5(2), and 5(2)(proviso) are incorrectly assigned to `Chapter XII: MISCELLANEOUS`. All Regulations 4–23 belong to Chapter II. One-time data correction in `data/schema/icdr_structure.json` before the full 301-regulation run.

### Fix 6 — `ICDR_10_iv` Wrong ID (1 rule, 1 FP)

Pass 1 returns `10(iv)` dropping the `(1)(d)` path. Add to `KNOWN_ID_RENAMES` in the post-processing script:

```
"ICDR_10_iv": "ICDR_10_1_d_iv"
```

### Fix 7 — Reg 12 Main Rule Missing (1 rule)

Only `ICDR_12_proviso` was extracted; Pass 2 merged the main prohibition into the proviso's text. Add to the Pass 2 system prompt: if a clause begins with the main regulatory obligation AND contains a `Provided that` sub-clause, always emit them as two separate objects. Never merge a main rule and its proviso.

### Fix 8 — Flag Suspicious False Positives (2 FP)

`ICDR_15_1_b_proviso` and `ICDR_16_1_a_proviso` appear in v6 output but not in the gold standard. Do not delete them — they may be valid 2025 amendment insertions. Instead, mark them `status = "needs_review"` in post-processing.

### Expected Impact

| Fix | Rules Recovered | FP Removed | Effort |
|---|---|---|---|
| 1 — Reg 6(3) carryover + anchoring | 12 | 0 | Medium |

| Fix | Rules Recovered | FP Removed | Effort |
|---|---|---|---|
| 2 — Inserted sub-reg `[(N)` pattern | 3 | 0 | Low |
| 3 — Reg 8 base-reg anchoring + explanation | 7 | 0 | Low |
| 4 — 1200-char cap + nested proviso split | 12 | 0 | Medium |
| 5 — icdr_structure.json data fix | 0 | 0 | Trivial |
| 6 — Rename `10_iv` → `10_1_d_iv` | 1 | 1 | Trivial |
| 7 — Reg 12 main+proviso separation | 1 | 0 | Low |
| 8 — Flag suspicious FPs | 0 | 2 | Trivial |
| **Total** | **~36** | **3** | |

**Projected v7:** TP ≈ 136, FN ≈ 7, FP ≈ 0–1 → **F1 ≈ 93–95%**

## Scoring Workflow for v7

After running v7 extraction, benchmark immediately:

```
python scripts/score_extraction.py \
    --extracted data/processed/rules_refactored_v7.jsonl \
    --gold      data/gold_standard/gold_standard_regs_4_23.jsonl \
    --baseline  data/processed/rules_refactored_v6.jsonl \
    --output    reports/v7_score_report.json
```

Exit code 0 = F1 ≥ 90% target met. The `--baseline` flag prints a delta column against v6 so regressions on any regulation are immediately visible.